



FasoShield

Plateforme nationale de protection contre les menaces mobiles

Moteur d'analyse d'APK · API de réputation · Gouvernance des signatures
Agent Android souverain · Partage de renseignement avec les CERT

DJIGUIMDE Cédric Severin
Dossier de présentation — 27/07/2026

1. Synthèse

FasoShield est une plateforme nationale de lutte contre les logiciels malveillants visant les téléphones mobiles, et plus précisément les comptes de monnaie électronique. Elle réunit un moteur d'analyse d'applications Android, une API de réputation interrogée par un agent mobile, une console d'analyse pour le CERT national, un circuit de validation des signatures et un canal de partage avec les CERT partenaires.

Le projet répond à un angle mort des antivirus commerciaux : leurs signatures visent les menaces mondiales, ils ne disposent d'aucun registre des applications financières locales, ils n'ont pas de règles sur les leurres rédigés en français, et leur télémétrie quitte le territoire. FasoShield renverse ces quatre points.

4	150	94 %	5/5	2
couches d'analyse	tests automatisés	couverture de code	phases livrées	formats CERT

Le choix structurant

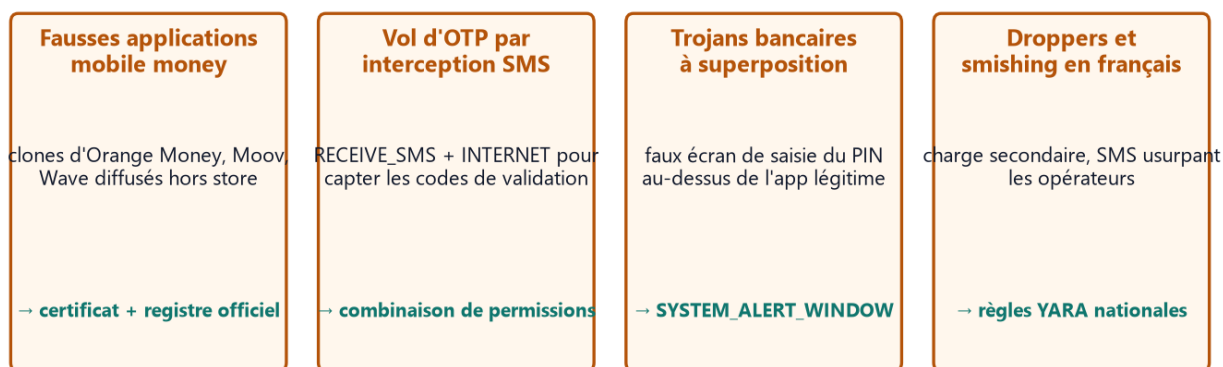
Aucun indicateur n'atteint les téléphones sans avoir été validé par deux analystes distincts. Un faux positif sur une application de monnaie électronique couperait des milliers d'utilisateurs de leurs fonds : c'est un risque plus grave qu'une détection manquée, et l'architecture est bâtie autour de ce constat.

Ce que contient ce document

2. Le problème — le paysage de menaces ouest-africain
3. L'architecture d'ensemble
4. Le moteur d'analyse en quatre couches
5. L'agent Android et le fonctionnement hors ligne
6. La console SOC et la séparation des identités
7. La gouvernance des signatures
8. Le partage de renseignement avec les CERT
9. Exploitation, montée en charge et déploiement
10. Sécurité et conformité
11. Qualité logicielle
12. Bilan et perspectives

2. Le problème

En Afrique de l'Ouest, le paiement mobile n'est pas un service annexe : c'est l'infrastructure financière dominante. Pour une grande partie de la population, le compte de monnaie électronique est le compte bancaire. Compromettre l'application, c'est donc compromettre directement l'épargne de la personne — sans intermédiaire, sans plafond de carte, sans procédure de contestation comparable à celle d'une banque.



Le paiement mobile est l'infrastructure financière dominante : compromettre l'application, c'est compromettre le compte.

Les quatre vecteurs dominants et le contrôle qui leur répond.

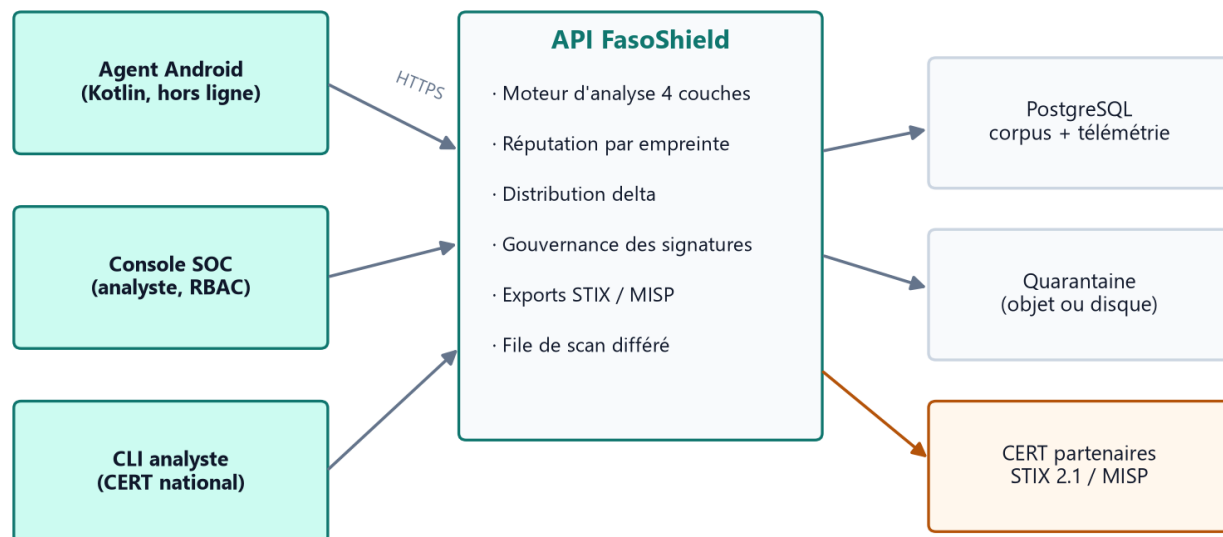
Ces campagnes partagent une caractéristique décisive : elles se diffusent **hors des magasins officiels**, par liens WhatsApp ou Telegram et par boutiques alternatives. C'est ce qui rend exploitable un signal que les antivirus génériques utilisent mal — la provenance de l'application.

Pourquoi les produits commerciaux couvrent mal ce terrain

Leurs bases de signatures sont alimentées par la télémétrie mondiale, où une campagne visant Orange Money au Burkina pèse statistiquement peu. Ils n'ont pas de registre des certificats de signature des applications financières locales, donc ils ne savent pas distinguer la vraie application de son clone. Et leurs règles de détection de hameçonnage sont écrites pour l'anglais.

3. Architecture d'ensemble

Trois clients, une API, un stockage national. L'agent mobile fonctionne hors ligne et n'interroge la plateforme que pour la réputation d'une empreinte et la synchronisation de ses signatures. La console sert le CERT. La ligne de commande couvre l'exploitation serveur, là où aucun navigateur n'est disponible.



Vue d'ensemble des composants et des flux.

Principes retenus

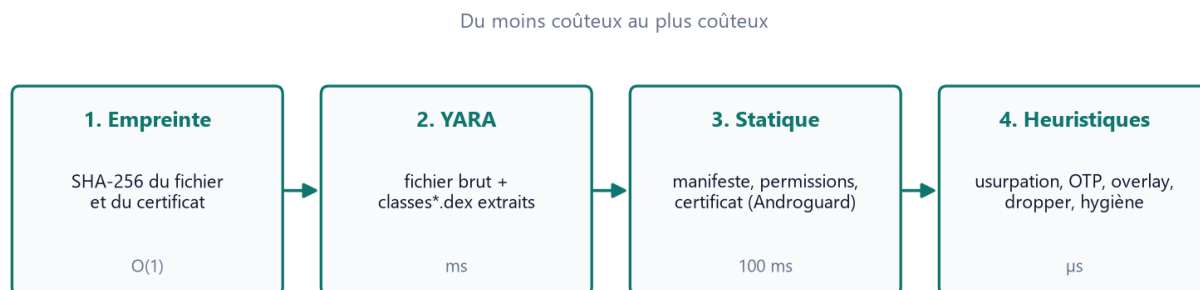
- **Souveraineté** — aucune dépendance à un service en nuage étranger. Le stockage objet est optionnel : la plateforme se déploie sur une infrastructure isolée, disque local compris.
- **Minimisation** — les champs identifiants n'existent pas dans le schéma de base de données. Ils ne peuvent donc pas être activés par un simple changement de configuration.
- **Dégradation maîtrisée** — l'agent protège l'appareil sans réseau ; l'analyse aboutit toujours à un rapport, même sur une archive corrompue.
- **Coût d'exploitation faible** — pas de courtier de messages, pas de cluster : la file de scan différée s'appuie sur la base de données.

Choix techniques

Composant	Technologie	Motif
API	FastAPI, SQLAlchemy 2	Contrats typés, documentation OpenAPI générée, portabilité SQLite → PostgreSQL
Analyse statique	Androguard	Lecture du manifeste binaire et du certificat de signature sans exécuter l'application
Détection de motifs	YARA	Format standard du métier ; les règles nationales sont lisibles et auditables par un tiers
Agent mobile	Kotlin, Room, WorkManager	Base locale, travaux en arrière-plan résistants au redémarrage
Authentification	scrypt, sessions serveur	Sans dépendance externe ; jetons révocables, stockés hachés

4. Le moteur d'analyse

Le pipeline enchaîne quatre couches, ordonnées du moins coûteux au plus coûteux. Chaque couche ajoute ses constats à un rapport commun ; le verdict final agrège les sévérités.



Le pipeline aboutit toujours : une archive corrompue ou un fichier non-APK produit quand même un rapport — aucune entrée ne peut faire taire le moteur.

Les quatre couches du moteur d'analyse.

Deux détails qui font la différence

Le DEX est compressé dans l'APK. Passer YARA sur le fichier brut ne voit rien du bytecode : les chaînes caractéristiques sont invisibles tant que `classes.dex` n'a pas été extrait. Le moteur extrait donc chaque entrée DEX et la scanne en mémoire, avec un plafond de 64 Mio et dix entrées au maximum pour borner le coût d'une archive hostile.

Le certificat de signature convainc mieux que l'empreinte du fichier. Un attaquant régénère son APK à volonté — l'empreinte change à chaque variante. La clé de signature, elle, est réutilisée sur toute une famille. La blocklist accepte donc les deux granularités, et un indicateur de certificat condamne les repacks que le moteur n'a jamais vus.

Couche heuristique

Règle	Signal	Sévérité
heur.cert_mismatch	Paquet officiel signé par une clé inconnue — clone repackagé	CRITIQUE
heur.package_lookalike	Nom de paquet très proche d'un paquet officiel	ÉLEVÉE
heur.brand_in_label	Marque financière dans le libellé d'un paquet non enregistré	ÉLEVÉE
heur.sms_exfiltration	Lecture des SMS entrants + accès réseau — vol d'OTP	ÉLEVÉE
heur.spyware_combo	Micro + réseau + localisation ou contacts	ÉLEVÉE
heur.overlay	Superposition d'écran — faux écran de saisie du PIN	MOYENNE
heur.dropper	Peut installer d'autres paquets	MOYENNE
heur.legacy_target_sdk	Cible un SDK antérieur aux permissions à l'exécution	MOYENNE

Un verdict qui s'explique

Le rapport ne renvoie pas un score opaque : chaque constat porte son identifiant de règle, sa sévérité, sa description et la preuve qui l'a déclenché. Un analyste peut contester une détection ligne par ligne — condition nécessaire pour qu'une publication d'indicateur soit revue sérieusement par un second analyste.

5. L'agent Android

L'agent analyse les applications installées **sur l'appareil** et n'envoie jamais d'APK sur le réseau. Il lit ce que PackageManager lui fournit — permissions, certificat, source d'installation, SDK cible — et applique les mêmes règles que le moteur serveur, avec les mêmes identifiants et les mêmes sévérités, pour que verdict local et verdict plateforme restent cohérents.

Fonctionnement hors ligne

- Base de signatures locale (Room), synchronisée en **delta** : l'agent ne télécharge que ce qui a été publié depuis sa dernière version.
- Détection des nouvelles installations par PACKAGE_ADDED, scan délégué à WorkManager.
- Scan périodique quotidien, ré-armé après redémarrage.
- Alerte à l'utilisateur avec le détail des raisons et la marche à suivre — l'agent **ne désinstalle jamais** lui-même.

La provenance comme filtre

C'est le raffinement que le serveur ne peut pas faire. L'appareil sait si une application est préinstallée ou vient d'un magasin officiel ; le serveur, qui ne voit qu'un fichier, l'ignore. Une application de messagerie installée depuis le Play Store lit légitimement les SMS pour remplir les codes de validation, et enregistre légitimement l'audio. La signaler noierait les vraies détections sous le bruit.

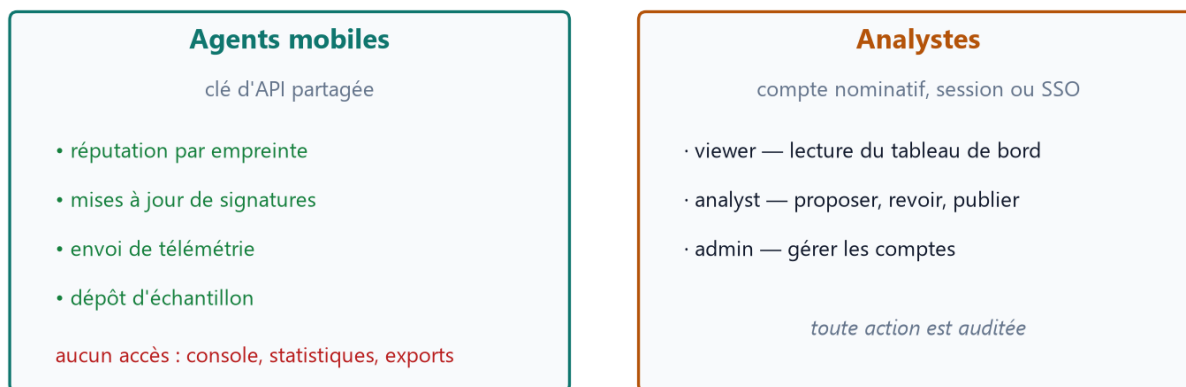
L'agent conditionne donc les heuristiques de permissions et d'hygiène à une provenance non fiable. Les contrôles d'usurpation et la blacklist, eux, s'appliquent toujours : un clone repackagé est malveillant quelle que soit sa source. Le nom du magasin installateur ne peut pas être falsifié par l'application installée, ce qui en fait un point d'ancrage solide.

Une correction issue de la mise au point sur appareil réel

Samsung Update Center déplace les applications système mises à jour vers /data/app et leur retire leurs indicateurs système. Sans traitement particulier, des applications constructeur parfaitement légitimes ressortaient comme installées hors magasin. L'installateur est reconnu comme source de confiance, et la détection d'origine système regarde aussi la partition d'installation.

6. Console SOC et séparation des identités

La console donne au CERT la vue nationale : détectations de terrain, répartition régionale, chronologie sur quatorze jours, menaces les plus fréquentes, file des signatures en attente de revue et journal d'audit.



Deux systèmes d'authentification, deux portées disjointes.

La distinction n'est pas cosmétique. Une clé d'agent est distribuée avec l'application : elle vit sur des milliers de téléphones, dont certains seront perdus, revendus ou compromis. La traiter comme un secret sérieux serait une fiction. Elle n'ouvre donc que les points d'entrée dont un téléphone a besoin — et ni la console, ni les statistiques, ni les exports.

Identité analyste

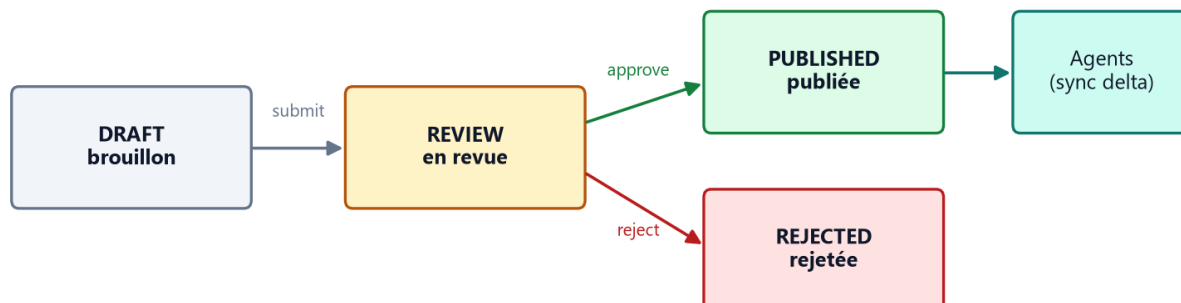
- Mot de passe haché en **scrypt** ($N = 2^{16}$, $r = 8$, soit 64 Mio de mémoire par calcul) : coûteux à casser sur GPU.
- par calcul) : coûteux à casser sur GPU.
- Session opaque stockée **uniquement sous forme d'empreinte SHA-256** — une copie de la base ne peut pas être rejouée comme session valide.
- Cookie HttpOnly, SameSite=Lax, Secure.
- Rôles **viewer** < **analyst** < **admin**. Un changement de mot de passe ou une désactivation révoque immédiatement les sessions.
- Réponse de connexion uniforme : mot de passe faux, compte inconnu et compte désactivé sont indiscernables, pour ne pas énumérer les opérateurs.
- Mode SSO par en-tête pour un déploiement derrière une passerelle OIDC, conditionné à l'isolement réseau de l'API.

Interface sans dépendance externe

La console n'utilise aucun CDN ni framework distant. Cela permet une politique de sécurité de contenu réellement stricte — `default-src 'none'` — avec un nonce régénéré à chaque réponse pour le script et la feuille de style embarqués. C'est aussi ce qui rend la console utilisable sur un réseau sans accès Internet sortant.

7. Gouvernance des signatures

C'est le cœur institutionnel du projet. Publier un indicateur revient à demander à des milliers d'appareils de considérer une application comme malveillante. Cette décision ne peut pas appartenir à une seule personne.



Règle des quatre yeux : l'auteur d'une proposition ne peut ni l'approuver ni la rejeter, même administrateur. Chaque transition est écrite au journal d'audit.

Le circuit de validation d'un indicateur.

Contrôle	Effet
Aucune écriture directe	La blocklist n'est jamais modifiée hors du circuit — ni par l'API, ni par la ligne de commande
Justification obligatoire	Une proposition sans éléments de preuve circonstanciés est refusée à la création
Règle des quatre yeux	L'auteur ne peut ni approuver ni rejeter sa propre proposition, même administrateur
Motif de rejet exigé	Un refus sans raison est impossible : la décision reste explicable
Journal d'audit	Proposition, soumission, publication et rejet sont consignés avec acteur, cible et adresse IP
Réversibilité	L'historique complet permet de retrouver qui a publié quoi, sur quelle base et quand

Une fois approuvé, l'indicateur entre dans la base de distribution et atteint les agents à leur synchronisation delta suivante. Les indicateurs de certificat sont stockés de manière à rester interrogeables par certificat, ce qui est la seule information dont dispose l'agent sans calculer l'empreinte de chaque APK installé.

```
$ fasoshield proposal list --status REVIEW
#12 REVIEW sha256 9f3c1a77b2e4d015... Trojan.FakeOM by alice / reviewed -

$ fasoshield proposal show 12
status : REVIEW
indicator : sha256 9f3c1a77b2e4d015...
proposed by : alice on 2026-07-27 09:14:22+00:00
justification :
Clone d'Orange Money diffusé par WhatsApp : capture le PIN saisi
et l'exfiltre vers un serveur de commande.
```

8. Partage avec les CERT partenaires

Les indicateurs publiés sont exportables dans les deux formats que les communautés CERT consomment réellement : **STIX 2.1**, la norme OASIS, et l'**événement MISP**, format des instances déployées dans les communautés africaines et européennes.

- Les identifiants d'objets sont dérivés de l'indicateur lui-même. Réexporter le même IOC met donc à jour l'objet chez le partenaire au lieu d'en créer un doublon — la différence entre un flux exploitable et un flux qui pollue.
- Un indicateur de certificat est exporté comme objet x509-certificate, pas comme empreinte de fichier : envoyer l'un pour l'autre enverrait le partenaire chercher un échantillon qui n'existe pas.
- Marquage TLP appliqué au bundle et à l'événement ; l'événement MISP est livré **non publié**, le CERT destinataire décide de sa diffusion.
- **Aucune donnée de télémétrie n'est exportée** : seules des empreintes de fichiers et de certificats franchissent la frontière.
- Chaque export est écrit au journal d'audit avec son auteur et son volume.

```
$ fasoshield intel stix -o bundle.json
412 indicators written to bundle.json
```

```
$ fasoshield intel misp --since 20260701000000 -o event.json
37 indicators written to event.json
```

9. Exploitation et déploiement

L'API est sans état et se réplique derrière un répartiteur de charge. La difficulté classique — analyser un gros APK sans bloquer une connexion HTTP — est traitée par une file de scan différé.

Au-delà d'un seuil de taille, le dépôt est mis en attente et le client reçoit un identifiant de travail qu'il interroge. La table des travaux est la file : un worker prend un travail par mise à jour conditionnelle, que la base sérialise pour nous. Plusieurs instances d'API, ou des workers dédiés, partagent donc la charge **sans courtier de messages** — un composant de moins à exploiter dans une infrastructure qui doit rester simple à maintenir.

Déploiement	Base	Quarantaine	Workers
Développement	SQLite	disque local	intégré à l'API
Serveur unique	PostgreSQL	disque local	intégré à l'API
Réparti	PostgreSQL	stockage objet (<i>obligatoire</i>)	conteneurs dédiés

Distribution de l'agent

La configuration de signature release lit un keystore externe au dépôt, jamais versionné. Si aucun keystore n'est fourni, le build produit un artefact **non signé** plutôt que de retomber silencieusement sur la clé de debug : une absence de signature se remarque immédiatement, un artefact signé en debug passerait inaperçu jusqu'aux utilisateurs. L'empreinte du certificat officiel doit être publiée pour que chacun puisse vérifier qu'il installe le vrai agent.

10. Sécurité et conformité

Le modèle de menaces retient cinq profils d'attaquants : l'opérateur de fraude qui cherche à ne pas être détecté, le concurrent qui voudrait faire inscrire une application légitime sur la blocklist, l'analyste interne malintentionné, l'attaquant réseau et l'application hostile installée à côté de l'agent.

Attaquant	Contrôle principal
Opérateur de fraude	Défense en profondeur à quatre couches ; le pipeline aboutit toujours, aucune entrée ne fait taire le moteur
Concurrent malveillant	Registre des applications officielles, heuristiques conditionnées à la provenance, quatre yeux avant publication
Analyste interne	Séparation auteur / relecteur, journal d'audit append-only, RBAC
Attaquant réseau	TLS, CSP stricte à nonce, en-têtes de sécurité, limitation de débit par appelant, CORS fermé
Application hostile locale	L'agent n'expose pas de composant exporté sensible et ne détient aucun secret réutilisable

Protection des données personnelles

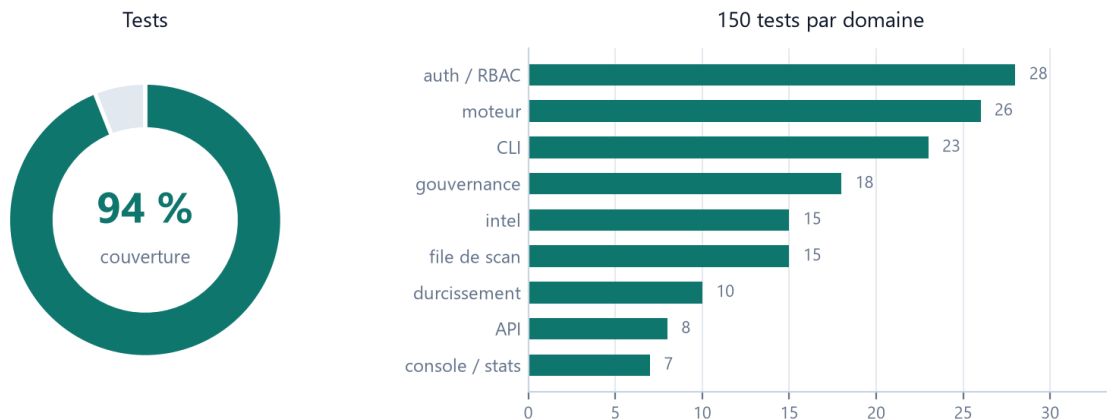
Le dispositif est soumis à analyse d'impact : il traite à grande échelle et observe en continu les applications installées sur l'appareil d'une personne. L'analyse a néanmoins été conduite sur un traitement conçu pour ne manipuler **aucune donnée directement identifiante**.

- Ne sont pas collectés, et n'existent pas dans le schéma : numéro de téléphone, IMEI, IMSI, identité, compte financier, contenu des SMS, contacts, position GPS, adresse IP de l'appareil.
- L'identifiant d'agent est un UUID généré sur l'appareil, sans lien avec un identifiant matériel ou d'abonné ; la région est déclarative et grossière.
- La console n'offre **aucune vue par appareil** : uniquement des agrégats. Elle ne peut pas servir à surveiller une personne.
- Le retrait du consentement à la télémétrie n'enlève aucune protection : l'agent continue de fonctionner hors ligne.

Le risque principal n'est pas la vie privée

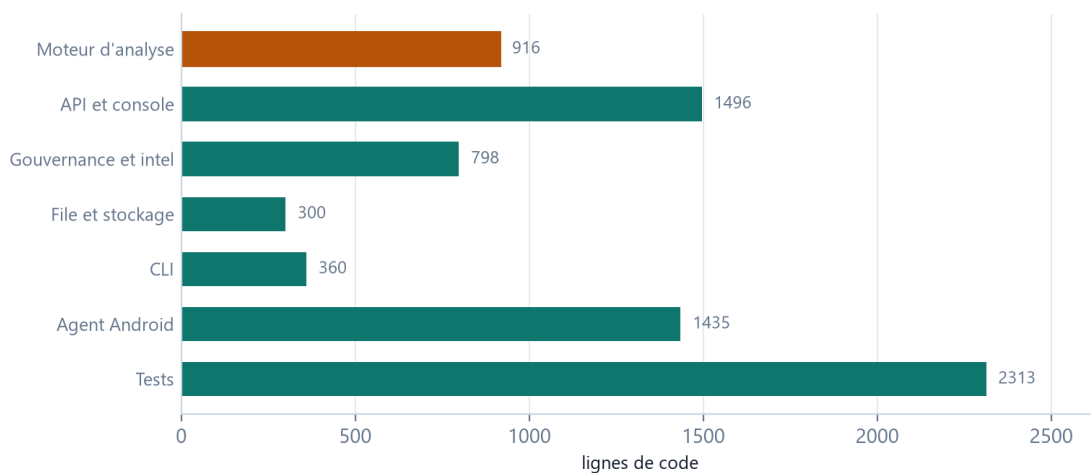
C'est le **faux positif**. Priver quelqu'un de son application de paiement, c'est le priver de son argent. Ce constat explique le registre des applications officielles avec épingle de certificat, le conditionnement des heuristiques à la provenance, la règle des quatre yeux, et le fait que l'agent alerte au lieu de désinstaller.

11. Qualité logicielle



Couverture et répartition des tests par domaine.

- **150 tests** automatisés, **94 %** de couverture, exécutés à chaque modification par l'intégration continue.
- Analyse statique ruff sans avertissement, SAST bandit sans constat, audit de dépendances pip-audit sans vulnérabilité connue.
- Deux chaînes d'intégration : une pour la plateforme Python, une pour l'agent Android, déclenchée sur toute modification du module mobile.
- Les tests portent sur les propriétés qui comptent — la règle des quatre yeux ne peut pas être contournée, une clé d'agent n'ouvre pas les statistiques, un jeton de session n'est jamais stocké en clair, un travail en échec ne tue pas le worker.



Répartition du code, mesurée sur le dépôt.

12. Bilan et perspectives



Reste ouvert : l'audit de sécurité externe — travail de tiers par nature — et la remise de la clé de signature nationale, préalable à la publication publique.

Les cinq phases du projet.

Les cinq phases prévues sont livrées : moteur d'analyse, API de plateforme, agent Android, console et gouvernance, durcissement et déploiement. La plateforme est fonctionnelle de bout en bout — un échantillon peut être soumis, qualifié, transformé en indicateur, validé par deux analystes, distribué aux agents et partagé avec un CERT partenaire.

Ce qui reste ouvert

- **L'audit de sécurité externe** — un travail de tiers par nature. Le périmètre attendu est énoncé : test d'intrusion authentifié, revue cryptographique, fuzzing du moteur, revue de l'agent et de la chaîne de build.
- **La remise de la clé de signature nationale**, préalable à toute publication publique de l'agent.
- **Les migrations de schéma** — le schéma est créé au démarrage, ce qui suffit aujourd'hui ; une exploitation pluriannuelle demandera un outil de migration. C'est un choix d'exploitation à trancher avant la première montée de version en production.

Extensions naturelles

- **Analyse dynamique** en bac à sable, pour les échantillons que l'obfuscation rend opaques à l'analyse statique.
- **Ingestion de flux partenaires** dans le sens entrant : la plateforme exporte aujourd'hui, elle pourrait consommer les flux MISP des CERT voisins.
- **Corrélation régionale** : détecter l'apparition d'une campagne à partir de la dérive statistique de la télémétrie, plutôt qu'à partir d'un signalement.

En une phrase

FasoShield démontre qu'une capacité antivirus souveraine, adaptée à un paysage de menaces local que les produits mondiaux couvrent mal, tient dans une architecture simple, exploitable par une petite équipe, et gouvernée de manière à ce qu'aucune personne seule ne puisse se tromper au nom de tout un pays.